



The Papyrus Handbook

A guide to Markdown books with Papyrus

milon/papyrus

The Papyrus Handbook

Markdown to PDF, EPUB, HTML sites, and KDP

Nuruzzaman Milon

Welcome

Thanks for opening **The Papyrus Handbook**. Use the sidebar to browse chapters, or continue from Introduction. The home page has the project banner, a **Downloads** link for PDF previews from GitHub, and links to GitHub and Packagist.

This guide is the companion for [milon/papyrus](https://github.com/milon/papyrus) — a PHP CLI that turns Markdown into PDF, EPUB, HTML, a multi-page site, and Amazon KDP exports. You are reading those same chapters as a Papyrus site.

How to use this guide

1. **Introduction** — what Papyrus is and the project layout
2. **Install** — Composer setup and `papyrus init`
3. **Configuration** — every `papyrus.php` option
4. **Writing** — Markdown, front matter, `pretoc`, asides, Mermaid, linting
5. **Themes** — PDF / HTML themes and fonts
6. **build** — all-in-one export, watch, caches
7. **build:pdf / epub / html / site / sample** — one chapter each
8. **kdp** and **kdp:*** — Kindle, print, covers, metadata
9. **Migration and CI** — ibis-next migration and GitHub Actions
10. **Command reference** — full CLI tables
11. **Downloads** — light and dark PDF previews from GitHub

Copyright © 2026 [Nuruzzaman Milon](#) / [Papyrus](#). Released under the same terms as the Papyrus project ([MIT](#)).

Table of Contents

Welcome	3
How to use this guide	4
Introduction	17
Who it is for	18
Project shape	19
Next steps	20
Install and project layout	21
Requirements	22
macOS (Homebrew)	22
npm / Linux	23
Check the toolchain	23

Install per project	24
Install globally	25
Scaffold a book	26
Layout	27
Configuration	28
Identity and paths	29
Page size and margins	31
Table of contents	32
Cover images	33
Running header	34
Fonts	35
Faces	36

Script rules	36
Site home (build:site)	38
Mermaid	40
Sample PDF	41
KDP	42
Break level	43
CommonMark hook	44
Writing content	45
Front matter	46
Pretoc chapters	47
Markdown features	49
Emphasis and code	49
Callouts	49

Page breaks	50
Mermaid	50
PHP fence linting	53
Themes and assets	54
PDF themes	55
HTML theme	56
EPUB styles	57
Covers and fonts	58
build	59
Options	60
What it builds	61
Failures	62

Validate first	63
Watch mode	64
Caching	65
Related chapters	66
build:pdf	67
Options	68
Output	69
Config that affects PDF	70
Parallel builds	72
Covers	73
build:epub	74

Options	75
Output	76
What goes in the package	77
Config that affects EPUB	78
Kindle upload	79
build:html	80
Options	81
Output	82
Config that affects HTML	83
Site vs HTML	84
build:site	85

Options	87
Output	88
Site config	89
Hosting this handbook	91
Mermaid on the site	92
build:sample	93
Options	94
Config	95
Output	97
kdp	98
Options	99

Enable flags	100
Typical artifacts	102
kdp:ebook	103
Options	104
Output	105
Cover	106
Metadata	107
Versus build:epub	108
kdp:print	109
Options	110
Output	111

Print config	112
Bleed	112
Margin presets (mm, before bleed)	112
Paper	113
Trim size	114
Cover	115
kdp:cover	116
Options	117
Sources and outputs	118
Example	119
kdp:metadata	120
Options	121
Output	122

Config	123
JSON shape	124
Migration and CI	126
Migrating from ibis-next	127
Continuous integration	128
Programmatic use	129
Checklist before release	130
Command reference	131
Build	132
KDP	133
Project tooling	134

Shared book options	135
Output filenames	136
Downloads	137
Full PDF	138
Sample PDF	139
Other previews	140

Introduction

Papyrus is a Composer package and CLI for authors who write books in Markdown and need more than one delivery format from a single tree of chapters.

One project layout yields:

- **PDF** — print interiors and shareable digital books
- **EPUB** — stores, e-readers, and Kindle upload
- **HTML** — a single file with light / dark mode
- **Site** — a multi-page static website (this handbook)
- **KDP helpers** — Kindle EPUB, print PDF, covers, metadata JSON

This handbook is itself a Papyrus book. The Markdown source lives in [examples/the-papyrus-handbook/](#) on GitHub. Prebuilt PDF, HTML, and site outputs are published from that example so you can read without installing anything.

Who it is for

- Authors shipping a technical or long-form book from Markdown
- Teams migrating off [ibis-next](#)
- Anyone who wants PDF and a hosted docs site from the same chapters

Project shape

Every book is a directory with:

Path	Role
<code>papyrus.php</code>	Title, trim, themes, Mermaid, samples, KDP
<code>content/</code>	Markdown chapters (natural filename order)
<code>assets/</code>	Your overrides (covers, banner, custom themes/fonts); bundled defaults used when absent
<code>export/</code>	Build outputs (usually gitignored)

Next steps

Install Papyrus (next chapter), run `papyrus doctor`, then `papyrus build` or `papyrus build:site`. Later chapters document every build and KDP command and the full `papyrus.php` option set.

Source and releases: github.com/milon/papyrus.

Install and project layout

Requirements

Required

- PHP 8.2+ with extensions `dom`, `gd`, `mbstring`, `zip`, and `zlib`
- Composer

Optional

Tool	Used by	Notes
<code>mmdc</code> (<code>@mermaid-js/mermaid-cli</code> or Homebrew <code>mermaid-cli</code>)	Mermaid diagrams	Needs Chrome/Chromium for Puppeteer
Chrome or Chromium	Mermaid CLI	Set <code>PUPPETEER_EXECUTABLE_PATH</code> if needed
<code>epubcheck</code>	<code>kdp:ebook</code>	Extra validation; skipped with a warning when absent

macOS (Homebrew)

```
brew install php composer
brew install mermaid-cli
brew install --cask google-chrome
brew install epubcheck
```

```
export PUPPETEER_EXECUTABLE_PATH="/Applications/Google  
Chrome.app/Contents/MacOS/Google Chrome"
```

npm / Linux

```
npm install -g @mermaid-js/mermaid-cli  
# Debian/Ubuntu example:  
sudo apt-get install -y chromium-browser  
export PUPPETEER_EXECUTABLE_PATH="$(command -v chromium-browser  
|| command -v google-chrome || command -v chromium)"
```

Install [epubcheck](#) and put it on **PATH**, or use **brew install epubcheck** on macOS.

Check the toolchain

```
php -m | grep -E 'dom|gd|mbstring|zip|zlib'  
mmdc --version  
epubcheck --version  
papyrus doctor
```


Install per project

In your book repository:

```
composer require milon/papyrus
```

```
vendor/bin/papyrus --version  
vendor/bin/papyrus init  
vendor/bin/papyrus doctor
```

Wire Composer scripts (example):

```
{  
  "scripts": {  
    "build": "papyrus build",  
    "build:pdf": "papyrus build:pdf --theme light,dark",  
    "build:epub": "papyrus build:epub",  
    "build:html": "papyrus build:html",  
    "build:site": "papyrus build:site",  
    "build:sample": "papyrus build:sample",  
    "build:kdp": "papyrus kdp",  
    "doctor": "papyrus doctor"  
  }  
}
```

Package page: packagist.org/packages/milon/papyrus.

Install globally

```
composer global require milon/papyrus
export PATH="$(composer global config bin-dir --absolute):$PATH"
papyrus list
```

Scaffold a book

```
papyrus init  
papyrus init -d my-book
```

`init` creates `papyrus.php`, `content/`, and an empty `assets/` directory. Papyrus uses bundled themes, CSS, and fonts by default. Publish those files into your project only when you want to customize them:

```
papyrus asset:publish  
papyrus asset:publish --only=themes,css
```

Use `--force` / `-f` to overwrite files during `init`, or with `asset:publish` to overwrite published assets. `--only` limits publishing to `themes`, `css`, and/or `fonts`.

Layout

Path	Role
<code>papyrus.php</code>	Book settings
<code>content/</code>	Markdown chapters
<code>assets/</code>	Your overrides: themes, CSS, covers, fonts, banner
<code>export/</code>	Built artifacts
<code>.papyrus/</code>	Incremental caches

Always run from the book root, or pass `-d / --dir`. Override where artifacts are written with `-e / --export` (default: `<book>/export`):

```
papyrus doctor -d /path/to/book
papyrus build --dir /path/to/book
papyrus build:site -d /path/to/book -e /path/to/out
```

Browse the Papyrus source on [GitHub](#) if you want to follow along with this handbook's own project under `examples/the-papyrus-handbook/`.

Configuration

All settings live in `papyrus.php`, which returns a PHP array.

Identity and paths

```
return [  
    'title' ⇒ 'My Book',  
    'subtitle' ⇒ 'A short subtitle',  
    'author' ⇒ 'Author Name',  
    'language' ⇒ 'en',  
    'themes' ⇒ ['light', 'dark'],  
  
    'content_dir' ⇒ 'content',  
    'assets_dir' ⇒ 'assets',  
    'export_dir' ⇒ 'export',  
];
```

Key	Default	Notes
<code>title</code>	<code>Untitled</code>	Drives the export filename slug
<code>subtitle</code>	<code>''</code>	
<code>author</code>	<code>''</code>	
<code>language</code>	<code>en</code>	EPUB / KDP metadata fallback
<code>themes</code>	<code>['light', 'dark']</code>	Each needs <code>assets/theme-{name}.html</code>
<code>content_dir</code>	<code>content</code>	Relative to the book root

Key	Default	Notes
<code>assets_dir</code>	<code>assets</code>	
<code>export_dir</code>	<code>export</code>	Overridden by <code>-e</code> / <code>--export</code>

The slug is the title lowercased with non-alphanumeric runs turned into `-`. This handbook is *The Papyrus Handbook*, so outputs look like `the-papyrus-handbook-light.pdf`.

Page size and margins

```
'document' => [  
  'size' => 'crown-quarto',  
  'margin_left' => 27,  
  'margin_right' => 27,  
  'margin_top' => 14,  
  'margin_bottom' => 14,  
],
```

List presets with **papyrus sizes**. Custom trim:

```
'document' => [  
  'format' => [152.4, 228.6], // width_mm, height_mm  
],
```


Table of contents

```
'toc' => [  
  'h1' => 0,  
  'h2' => 0,  
  'h3' => 1,  
],
```

Values follow mPDF **h2toc** conventions used by Papyrus (**0** include, **1** exclude).

Cover images

```
'cover' => [  
  'image' => 'cover.png',  
  'light' => 'cover-light.png',  
  'dark' => 'cover-dark.png',  
  // 'width' => 188.976, // mm; defaults to page width  
  // 'height' => 246.126, // mm; defaults to page height  
],
```

Paths are relative to **assets/**. Missing per-theme covers fall back to **cover.image**. KDP print PDFs omit the cover page entirely.

Running header

```
'header' => [  
  'style' => 'font-style: italic; text-align: right; border-  
bottom: solid 1px #808080;',  
],
```

Pretoc chapters suppress folios differently from body chapters.

Fonts

Declare faces under `assets/fonts/` and optional **script routing** (PDF only). First matching **script** rule wins.

```
'fonts' => [
  'default' => 'librelibertine', // optional; else first face,
  else mPDF default
  'faces' => [
    [
      'name' => 'librelibertine',
      'regular' => 'LinLibertine_R.ttf',
      'bold' => 'LinLibertine_RB.ttf',
      'italic' => 'LinLibertine_RI.ttf',
      'bold_italic' => 'LinLibertine_RBI.ttf',
    ],
    [
      'name' => 'notosansbengali',
      'regular' => 'NotoSansBengali-Regular.ttf',
      'otl' => true, // OpenType layout – needed for
complex scripts
    ],
  ],
  'script' => [
    // When mPDF tags a run as Bengali, use this face instead
of the default
    ['match' => ['bn', 'ben', 'bengali'], 'face' =>
'notosansbengali'],
  ],
],
```

Faces

Each face needs a **name** and a **regular** file that exists under **assets/fonts/**. Optional keys: **bold**, **italic**, **bold_italic**, and **otl** (sets mPDF **useOTL** when true — common for code fonts and Indic / Arabic scripts).

Papyrus merges these into mPDF's **fontdata** when building PDF (**FontRegistry** → **MpdfFactory**).

Script rules

fonts.script is a list of { **match**, **face** } rules used only for **PDF**. They do not change EPUB or HTML typography.

When any applicable rule exists, Papyrus turns on mPDF's **autoScriptToLang** and **autoLangToFont**, and installs a custom **languageToFont** resolver (**ScriptLanguageToFont**). mPDF then:

1. Detects script runs in the HTML (e.g. Bengali glyphs).
2. Tags them with a language / script code (e.g. **bn**, or ***-beng**).
3. Asks Papyrus which face to use; the first rule whose **match** list hits that code wins.

match aliases are lowercased. Built-in aliases include:

Aliases you can list	Resolves as
bn , ben , beng , bengali	Bengali

Aliases you can list	Resolves as
<code>ar, arab, arabic</code>	Arabic
<code>hi, hin, deva, devanagari, hindi</code>	Devanagari
any 4-letter ISO script code	that script as-is

The `face` string must be a name from `fonts.faces` and that face must actually load (regular file present). Rules pointing at a missing face are skipped (`FontRegistry::applicableScriptRules()`). Unmatched languages fall through to mPDF's built-in `LanguageToFont`.

Example: body text in Latin uses `librelibertine`; a paragraph containing বন্ধন is auto-tagged Bengali and rendered with `notosansbengali` if that face is registered.

Notice: Without both a registered face and a matching `script` rule, non-Latin runs often show as tofu (missing glyphs) in the default font.

Site home (**build:site**)

```
'site' => [
  'banner' => 'banner.jpg',
  'lead' => 'A one-line pitch for the home page.',
  'cname' => 'docs.example.com',
  'base_path' => '/my-repo', // project Pages path; omit for
  custom domains at /
  'links' => [
    ['label' => 'Downloads', 'chapter' => '19-downloads.md'],
    ['label' => 'Source on GitHub', 'url' =>
'https://github.com/you/your-book'],
    ['label' => 'Issues', 'url' =>
'https://github.com/you/your-book/issues'],
  ],
],
```

Key	Default	Notes
banner	auto banner.jpg / banner.png if present	Under assets/
lead	unset	Home page pitch
cname	unset	Custom domain; writes a CNAME file in the site root for GitHub Pages; absolute sitemap.xml / robots.txt Sitemap URL

Key	Default	Notes
<code>base_path</code>	unset (site at /)	Path prefix for project GitHub Pages (e.g. <code>/my-repo</code>); injects <code><base href></code> ; also prefixes sitemap locs
<code>links</code>	unset	Explicit home-page links; each item needs <code>label</code> plus either <code>url</code> or <code>chapter</code>

`chapter` matches a chapter source name like `19-downloads.md`, `19-downloads`, or a full relative source path, and links to that generated page. `repository` still works as a legacy fallback that auto-adds GitHub, Packagist, and Issues links when `links` is not set. Chapters are never linked automatically; add them explicitly through `links`.

Mermaid

```
'mermaid' => [  
  'enabled' => true,  
  'format' => 'svg',           // svg | png  
  'theme' => 'auto',          // auto = book colours; or  
                               default/dark/forest/neutral  
  'max_width_mm' => 130,  
  // 'command' => '/usr/local/bin/mmdc', // optional override  
],
```

Key	Default	Notes
enabled	false	Requires mmdc on PATH (or command)
format	svg	Anything other than png becomes svg
theme	auto	Book-matched palettes for PDF; HTML/site embed light + dark
max_width_mm	130	PDF figure width cap
command	auto-resolve mmdc	Explicit CLI path

Sample PDF

```
'sample' => [  
  'ranges' => [  
    ['from' => 1, 'to' => 3],  
  ],  
  'chapters' => [  
    '01-introduction.md',  
  ],  
],  
'sample_notice' => 'This is a sample from My Book.'
```

Ranges are 1-based inclusive pages of the finished theme PDF. **chapters** lists content filenames (or stems) to include in full. Notice text also accepts **sample.notice** / **sample.text** as fallbacks. See the **build:sample** chapter.

KDP

```
'kdp' => [  
  'ebook' => [  
    'enabled' => true,  
    'cover' => 'cover-ebook.jpg',  
  ],  
  'print' => [  
    'enabled' => true,  
    'bleed_mm' => 3,  
    'margin_preset' => 'recommended', // or minimum  
    'paper' => 'white',  
  ],  
  'metadata' => [  
    'description' => 'Bookstore blurb...',  
    'keywords' => ['laravel', 'filament'],  
    'language' => 'en',  
  ],  
],
```

Full option tables live in the **kdp** and **kdp:*** chapters. **papyrus build** only runs KDP when ebook or print is enabled; **papyrus kdp** errors if neither is.

Break level

```
'break_level' => 2,
```

Automatic page breaks before headings (**1** = H1, **2** = H1 and H2). Explicit **[break]** markers always insert a break.

CommonMark hook

```
'configure_commonmark' => function  
(\League\CommonMark\Environment\Environment $environment): void {  
    // register custom extensions  
},
```

Writing content

Chapters are Markdown under `content/`, loaded in natural filename order (`00-...`, `01-...`, `10-...`).

Front matter

```
---
title: My chapter
pretoc: true
---

Body starts here.
```

Key	Meaning
<code>title</code>	Chapter title for EPUB / site sidebar / running headers
<code>pretoc</code>	<code>true</code> places the chapter before the PDF table of contents
<code>draft</code>	<code>true</code> omits the chapter from PDF, EPUB, HTML, site, sample, and KDP builds

Accepted truthy values for `pretoc` and `draft`: `true`, `"true"`, `1`, `"1"`.

Pass `--include-drafts` on any book build command (or `watch`) when you want draft chapters in the output. `doctor` lists drafts and warns if `site.links` points at one.

Pretoc chapters

In the PDF, Papyrus splits the book into three bands around the theme's TOC marker (`<!-- PAPYRUS:TOC -->`):

1. **Pretoc** chapters (`pretoc: true`) — title page material, copyright, dedication, welcome, etc.
2. **Table of contents** — generated from body headings (see `toc` in config)
3. **Body** chapters — everything else, with folios and the running header

Pretoc chapters:

- Appear in EPUB, HTML, and the site like any other chapter
- Do **not** get normal body folios the same way (front matter styling)
- Are omitted from the auto-generated PDF TOC as body entries
- Still sort by filename — use a low prefix such as `00-` so they come first

This handbook's Welcome page is the example:

```
---
title: Welcome
pretoc: true
---

# Welcome

...
```

File: `content/00-welcome.md`. Introduction and later chapters omit

`pretoc` (or set it false) so they sit after the TOC in the PDF.

You can have several pretoc files (`00-copyright.md`, `00-dedication.md`, `00-welcome.md`); all of them render before the TOC, in filename order.

Markdown features

CommonMark + GFM, fenced code highlighting, and book extras.

Emphasis and code

```
Hello, **world**.
```

```
`inline code`
```

```
```php
$user = User::factory()→create();
```
```

Callouts

```
> {notice} Something helpful.
```

```
> {warning} Something risky.
```

```
 ::: note
A note aside.
 :::

 ::: warning
A warning aside.
 :::
```

Page breaks

```
[break]
```

Mermaid

Write a **mermaid** fence in your chapter. Papyrus runs **mmdc** at build time and replaces the fence with an SVG (or PNG) figure in PDF, EPUB, HTML, and site exports.

Source:

```
```mermaid
flowchart TB
 subgraph Authoring
 MD[Markdown chapters]
 CFG[papyrus.php]
 ASSETS[Themes and fonts]
 end

 subgraph Papyrus
 CONVERT[BookConverter]
 MERMAID[MermaidRenderer]
 PDF[build:pdf]
 EPUB[build:epub]
 HTML[build:html]
 SITE[build:site]
 end

 MD --> CONVERT
 CFG --> CONVERT
 ASSETS --> PDF
 ASSETS --> HTML
 ASSETS --> SITE
 CONVERT --> MERMAID
 MERMAID --> PDF
 MERMAID --> EPUB
 MERMAID --> HTML
 MERMAID --> SITE
```
```

Rendered output:



Enable in `papyrus.php` (theme defaults to `auto` — book colours; HTML and site embeds both light and dark variants):

```
'mermaid' => [  
    'enabled' => true,  
],
```

Optional knobs: `format` (`svg` / `png`), `theme` (`auto` or a Mermaid stock theme like `default` / `dark` / `forest`), `max_width_mm`.

Requires `@mermaid-js/mermaid-cli` (`mmdc`) on `PATH`. Diagrams cache under `.papyrus/cache/mermaid`.

PHP fence linting

```
papyrus lint
papyrus lint --fix
papyrus lint --max-width=66
```

| Option | Short | Default | Meaning |
|-------------------------------|----------------------|-----------------|--|
| <code>--fix</code> | <code>-f</code> | off | Apply auto-fixes for open tags and comment runs |
| <code>--max-width</code> | — | <code>66</code> | Maximum line width for PHP fences |
| <code>--dir / --export</code> | <code>-d / -e</code> | book defaults | Shared book options (<code>--export</code> unused for lint) |

Themes and assets

Everything visual can live under **assets/**, but it does not have to. New projects from **papyrus init** start with an empty **assets/** directory and use the bundled Papyrus defaults: Linux Libertine for body and headings, 0xProto for code, github-gist highlight colors, notice / tip / caution asides, and matching font files.

Publish the bundled assets into your project when you want to edit them:

```
papyrus asset:publish
papyrus asset:publish --only=themes,css
papyrus asset:publish --only=fonts --force
```

--only accepts **themes**, **css**, and/or **fonts**. Omit it to publish everything.

PDF themes

For each name in **themes**, Papyrus looks for **assets/theme-{name}.html** first, then falls back to the bundled copy. Split the theme on the TOC marker:

```
<!-- PAPYRUS:TOC -->
```

Papyrus writes pretoc chapters, then the TOC, then body chapters around that marker. A legacy **<!-- IBIS:TOC -->** marker is still accepted. **migrate-ibis** rewrites that marker only in **your assets/theme*.html** files, not in the bundled defaults.

Typical theme responsibilities: **@page** / typography, code blocks, callout colors, title page, and running header styles (often paired with **header.style** in config).

HTML theme

`theme-html.html` is a full HTML document with placeholders `{{ $title }}`, `{{ $subtitle }}`, `{{ $author }}`, and `{{ $body }}`. The default template mirrors the light PDF theme (same typefaces, colors, code blocks, and callouts), with a fixed moon/sun icon toggle that switches to the dark PDF palette. The reading column follows Tailwind screen widths (640 → 1536px). `@font-face` rules in the theme use `../assets/fonts/`. For `build:html`, Papyrus embeds those font files as base64 data URIs so the export is self-contained. For `build:site`, fonts are copied into the site `assets/fonts/` folder.

```
papyrus build:html
papyrus build:site
```

`build:site` writes `export/<slug>-site/` with one HTML page per chapter, a Home index, shared CSS/JS (fonts copied into the site), a collapsible chapter sidebar, and the same light/dark toggle as the single-file HTML export. That folder is what this handbook deploys to GitHub Pages.

EPUB styles

`style.css` (and optional `highlight.codeblock.min.css`) are loaded from your project `assets/` first, then from Papyrus's bundled defaults, and ship inside the EPUB. Prefer simple `pre/code` rules for e-ink readers.

Covers and fonts

Place cover images and font files under `assets/` (fonts usually in `assets/fonts/`). Reference them from `cover` and `fonts.faces` in `papyrus.php`.

This handbook keeps only book-specific assets in `assets/` — `cover.jpg` for the PDF cover and `banner.jpg` for the site home page. Themes, CSS, and fonts come from Papyrus's bundled defaults; run `papyrus asset:publish` when you want local copies to customize. The wide banner image in the Papyrus repo README lives at [assets/papyrus-banner.jpg](#).

build

papyrus build is the all-in-one export command. From the book root (or with **-d**), it runs every configured PDF theme, then EPUB, then single-file HTML, then every **enabled** KDP output. Site and sample builds are opt-in.

```
papyrus build
papyrus build --with-site --with-sample
papyrus build -d /path/to/book
papyrus build -d examples/the-papyrus-handbook -e docs --with-site --with-sample
```

Options

| Option | Short | Default | Meaning |
|-------------------------------|-----------------|---|--|
| <code>--dir</code> | <code>-d</code> | current directory | Book root (must contain <code>papyrus.php</code>) |
| <code>--export</code> | <code>-e</code> | <code>export_dir</code> from config (<code>export</code>) | Where artifacts are written |
| <code>--with-site</code> | | off | Also run <code>build:site</code> |
| <code>--with-sample</code> | | off | Also run <code>build:sample</code> for every theme |
| <code>--include-drafts</code> | | off | Include chapters with <code>draft: true</code> |

Global Symfony options also apply (`-v` / `--verbose`, `-q`, `--no-ansi`, ...).
With `-v`, Papyrus prints third-party notices from mPDF and the EPUB zipper.

What it builds

| Step | Always? | Output |
|---|---|---|
| PDF
(each themes entry) | yes | <code>export/<slug>-<theme>.pdf</code> |
| EPUB | yes | <code>export/<slug>.epub</code> |
| HTML | yes | <code>export/<slug>.html</code> |
| KDP
ebook /
print /
covers /
metadata | only if
<code>kdp.ebook.enabled</code>
or
<code>kdp.print.enabled</code> | see KDP chapters |
| Site | only with <code>--with-site</code> | <code>export/<slug>-site/</code> |
| Sample
PDF | only with <code>--with-sample</code> | <code>export/sample-<slug>-<theme>.pdf</code> |

Slug comes from **title** (lowercased, non-alphanumeric → -).

watch invokes **build** on each change. Pass `--with-site` and/or `--with-sample` to include those outputs in every rebuild. Pass `--include-drafts` to keep **draft: true** chapters in the rebuild.

Failures

Each step is independent. If one format fails, Papyrus continues with the others and exits non-zero if any step failed. When no KDP outputs are enabled, the KDP block is skipped silently (unlike `papyrus kdp`, which errors).

Validate first

```
papyrus doctor  
papyrus build
```


Watch mode

Rebuild on changes to `papyrus.php`, `content/`, and `assets/`:

```
papyrus watch
papyrus watch --interval 3
papyrus watch --with-site --with-sample
```

| Option | Short | Default | Meaning |
|-------------------------------|----------------------|----------------------------|--------------------------------------|
| <code>--interval</code> | <code>-i</code> | 2 | Poll interval in seconds (minimum 1) |
| <code>--dir / --export</code> | <code>-d / -e</code> | same as <code>build</code> | Passed through to each rebuild |
| <code>--with-site</code> | | off | Also run <code>build:site</code> |
| <code>--with-sample</code> | | off | Also run <code>build:sample</code> |

Caching

Chapter HTML caches under `.papyrus/cache/markdown`. Mermaid figures use `.papyrus/cache/mermaid`. Delete `.papyrus/` for a cold rebuild.

Related chapters

The next chapters cover each `build:*` command and every KDP command in turn.

build:pdf

Build one PDF per theme (or a subset). Each theme needs `assets/theme-{name}.html` and an entry in `themes`.

```
papyrus build:pdf
papyrus build:pdf --theme light
papyrus build:pdf --theme light,dark --parallel
papyrus build:pdf -d /path/to/book -e /path/to/out
```

Options

| Option | Short | Default | Meaning |
|-------------------------|-----------------|-----------------------|-----------------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |
| <code>--theme</code> | <code>-t</code> | all configured themes | Comma-separated theme names |
| <code>--parallel</code> | <code>-p</code> | off | One process per theme |

Output

```
export/<slug>-<theme>.pdf
```

Example for this handbook: `the-papyrus-handbook-light.pdf`.

Config that affects PDF

| Key | Role |
|--|---|
| <code>themes</code> | Which theme names to build by default |
| <code>document.size / document.format</code> | Trim size |
| <code>document.margin_*</code> | Margins in millimetres |
| <code>cover</code> | Cover image(s) under <code>assets/</code> |
| <code>header.style</code> | Running header CSS |
| <code>toc</code> | Which heading levels enter the TOC |
| <code>break_level</code> | Auto page breaks before headings |
| <code>fonts</code> | Faces and script routing |
| <code>mermaid</code> | Diagram rendering |

List trim presets with `papyrus sizes`. Custom trim:

```
'document' => [  
  'format' => [152.4, 228.6], // width_mm, height_mm  
],
```


Parallel builds

`--parallel` / `-p` spawns one PHP process per theme. Useful when `themes` has both `light` and `dark`. Sequential is the default (and what `papyrus build` uses).

Covers

```
'cover' => [  
  'image' => 'cover.png',          // fallback for every theme  
  'light' => 'cover-light.png',    // optional per-theme  
  'dark' => 'cover-dark.png',  
  // 'width' / 'height' - mm; default to page size  
],
```

Paths are relative to **assets/**. Missing per-theme covers fall back to **cover.image**.

build:epub

Build an EPUB3 package from the same Markdown chapters as PDF and HTML.

```
papyrus build:epub  
papyrus build:epub -d /path/to/book  
papyrus build:epub -e /path/to/out
```

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|----------------------|------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |

Output

```
export/<slug>.epub
```

What goes in the package

- Chapter HTML derived from `content/`
- `assets/style.css` and optional `highlight.codeblock.min.css`
- Embedded images referenced from chapters
- Cover from `cover` (theme fallback → `cover.image`)
- Metadata from `title`, `subtitle`, `author`, and `language`

Config that affects EPUB

| Key | Role |
|---|--|
| <code>title</code> , <code>subtitle</code> ,
<code>author</code> | Package identity |
| <code>language</code> | EPUB language (default <code>en</code>) |
| <code>cover</code> | Cover image under <code>assets/</code> |
| <code>mermaid</code> | Pre-rendered figures |
| <code>fonts</code> | Faces are PDF-oriented; EPUB relies on CSS |

Prefer simple `pre` / `code` rules in `assets/style.css` for e-ink readers.

Kindle upload

For Amazon KDP's Kindle pipeline, prefer `kdp:ebook` (validates and names the file for upload). `build:epub` is the general-purpose store / archive build.

build:html

Build a **single-file** HTML book with light / dark mode.

```
papyrus build:html  
papyrus build:html -d /path/to/book -e docs
```

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|----------------------|------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |

Output

```
export/<slug>.html
```

Requires `theme-html.html` (project `assets/` or bundled default).

Placeholders in the theme:

| Placeholder | Source |
|-------------------------------|-----------------------|
| <code>{{ \$title }}</code> | <code>title</code> |
| <code>{{ \$subtitle }}</code> | <code>subtitle</code> |
| <code>{{ \$author }}</code> | <code>author</code> |
| <code>{{ \$body }}</code> | Rendered chapters |

The default stub theme mirrors the light PDF palette, with a moon/sun toggle that switches to the dark PDF colours. The reading column follows Tailwind screen widths (640 → 1536px).

`@font-face` rules that point at `../assets/fonts/` are embedded as base64 data URIs at build time, so the single HTML file is self-contained even when fonts come from Papyrus's bundled defaults.

Config that affects HTML

| Key | Role |
|---|--|
| <code>title</code> , <code>subtitle</code> ,
<code>author</code> | Document chrome |
| <code>mermaid</code> | Figures; <code>theme</code> $\Rightarrow$ <code>auto</code> embeds light + dark SVGs |
| <code>fonts</code> | Loaded via the HTML theme's <code>@font-face</code> rules |

Site vs HTML

| Command | Result |
|-------------------------|---|
| <code>build:html</code> | One self-contained <code>.html</code> file |
| <code>build:site</code> | Multi-page folder with sidebar, Home, shared CSS/JS |

Use `build:html` for emailing or archiving a single document; use `build:site` for hosting.

build:site

Build a multi-page static site: Home index, one HTML page per chapter, sidebar navigation, light / dark mode, and Prev / Next links.

```
papyrus build:site
papyrus build:site -d examples/the-papyrus-handbook -e docs
```

papyrus build does **not** run this by default — pass **--with-site** on **build**, or call **build:site** directly.

Preview the site (including popup search) with PHP's built-in server:

```
papyrus serve
papyrus serve --build
papyrus serve --port 8080
papyrus serve -s docs/the-papyrus-handbook-site
papyrus serve -d examples/the-papyrus-handbook -e docs --build
```

By default **serve** reads **export/<slug>-site/**. Pass **--site** / **-s** to point at another folder (for example the handbook under **docs/**) — no book project is required in that case. **--export** / **-e** still changes the default parent (**<export>/<slug>-site**). Pass **--build** to run **build:site** first (into that same site directory; needs **-d** / a book root). If **site.base_path** is set on a loaded project, the printed URL includes that prefix so **<base href>**

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|----------------------|--------------------------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Parent directory for the site folder |

Output

```
export/<slug>-site/
  index.html
  404.html
  <chapter-slug>.html
  sitemap.xml
  robots.txt
  .nojekyll
  CNAME                # when site.cname is set
  assets/site.css
  assets/site.js
  assets/search.json
  assets/fonts/...
  assets/<banner>      # when configured
```

Point any static host (GitHub Pages, Netlify, S3, ...) at that folder. **.nojekyll** tells GitHub Pages not to run Jekyll. **CNAME** (from **site.cname**) sets a custom domain. GitHub Pages and Netlify serve **404.html** for missing URLs. Press **/** or the topbar search button for popup search (**↑/↓** moves through results, Enter opens the hit, Escape closes). **sitemap.xml** uses **https://{cname}** when **site.cname** is set, otherwise the **site.base_path** prefix. **robots.txt** points at the sitemap when a CNAME is configured. Each **##** heading gets an id and a **#** permalink; search hits for those headings open the chapter at that section.

Site config

Optional **site** block in **papyrus.php**:

```
'site' => [
    'banner' => 'banner.jpg',           // under assets/; auto-
detects banner.jpg / banner.png
    'lead' => 'A one-line pitch for the home page.',
    'cname' => 'docs.example.com',      // GitHub Pages custom
domain
    'base_path' => '/my-repo',          // project Pages under
github.io/my-repo/; omit with cname
    'links' => [
        ['label' => 'Downloads', 'chapter' => '19-downloads.md'],
        ['label' => 'Source on GitHub', 'url' =>
'https://github.com/you/your-book'],
    ],
],
```

| Key | Default | Meaning |
|---------------|--|--|
| banner | banner.jpg
then
banner.png if
present | Hero image on Home |
| lead | unset | Short pitch under the title on
Home |

| Key | Default | Meaning |
|------------------------|---------|--|
| <code>cname</code> | unset | Writes <code>CNAME</code> in the site root for a GitHub Pages custom domain |
| <code>base_path</code> | unset | URL prefix for project Pages (writes <code><base href="/prefix/"></code>) |
| <code>links</code> | unset | Home-page links; each item needs <code>label</code> plus either <code>url</code> or <code>chapter</code> |

Nothing is inferred from chapter titles. If you want a Downloads link on Home, add it explicitly with `['label' => 'Downloads', 'chapter' => '19-downloads.md']`.

Use `base_path` when the site is not at the domain root (for example `https://user.github.io/my-repo/`). Leave it unset when using a custom domain via `cname`.

Hosting this handbook

In the Papyrus repository, pushes to **main** rebuild the handbook site and publish it to **gh-pages** via **.github/workflows/pages.yml**. Locally:

```
composer build:handbook
# or just:
papyrus build:site -d examples/the-papyrus-handbook -e docs
```

Mermaid on the site

With `mermaid.theme => auto`, each diagram embeds light and dark SVG variants; CSS swaps them with `data-theme`. Diagrams stay within the reading column (same max-width ladder as the HTML theme).

build:sample

Build a **sample** PDF for marketing downloads or review copies. Select content with page ranges, whole chapters by filename, or both.

```
papyrus build:sample
papyrus build:sample --theme light
papyrus build:sample -t light,dark
```

papyrus build does **not** run this by default — pass **--with-sample**, or call **build:sample** directly.

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|-----------------------|-----------------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |
| <code>--theme</code> | <code>-t</code> | all configured themes | Comma-separated theme names |

Config

```
'sample' => [
  'ranges' => [
    ['from' => 1, 'to' => 4],
    ['from' => 10, 'to' => 12],
  ],
  'chapters' => [
    '01-introduction.md',
    '04-writing-content', // stem or basename also works
  ],
  // Optional notice text (also accepted as sample.notice /
  sample.text):
  // 'notice' => 'This is a sample...',
],

'sample_notice' => 'This is a sample from My Book.',
```

| Key | Meaning |
|------------------------------|---|
| <code>sample.ranges</code> | List of {from, to} 1-based inclusive page ranges against the finished PDF |
| <code>sample.chapters</code> | Chapter files to include in full (config order). Match by source path, basename (01-intro.md), or stem (01-intro). Rendered as body pages only — no extra cover, title page, or TOC |

| `sample_notice` | Extra notice page (preferred) | | `sample.notice` /
`sample.text` | Fallbacks if `sample_notice` is empty |

Invalid ranges are dropped. Unknown chapter names fail the build.
`build:sample` requires at least one of `ranges` or `chapters`.

When both are set, page-range slices from the full theme PDF come first, then chapter pages (in config order).

Output

```
export/sample-<slug>-<theme>.pdf
```

When a notice is set, it is appended after the selected content.

kdp

papyrus kdp builds every **enabled** Amazon KDP artifact in one shot: Kindle EPUB (if enabled), print interior PDF (if enabled), cover asset copies, and the metadata JSON sidecar.

```
papyrus kdp  
papyrus kdp -d /path/to/book -e /path/to/out
```

Unlike **papyrus build**, this command **fails** when neither **kdp.ebook.enabled** nor **kdp.print.enabled** is true.

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|----------------------|------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |

Print uses the **first** configured theme (same as `kdp:print` without `--theme`).

Enable flags

```
'kdp' => [
  'ebook' => [
    'enabled' => true,
    'cover' => 'cover-ebook.jpg',
  ],
  'print' => [
    'enabled' => true,
    'bleed_mm' => 3,
    'margin_preset' => 'recommended', // or minimum
    'paper' => 'white',                // metadata only (e.g.
cream)
  ],
  'metadata' => [
    'description' => 'Bookstore blurb...',
    'keywords' => ['papyrus', 'markdown'],
    'language' => 'en',
  ],
],
```

| Key | Default | Notes |
|----------------------------|--------------------|----------------------------------|
| <code>ebook.enabled</code> | <code>false</code> | Gates ebook EPUB |
| <code>ebook.cover</code> | unset | Asset under <code>assets/</code> |
| <code>print.enabled</code> | <code>false</code> | Gates print PDF |

| Key | Default | Notes |
|----------------------------------|----------------------|-------------------------------------|
| <code>print.bleed_mm</code> | 3 | Added to trim and each margin |
| <code>print.margin_preset</code> | recommended | recommended or minimum |
| <code>print.paper</code> | white | Recorded in metadata only |
| <code>metadata.*</code> | see metadata chapter | Sidecar + EPUB description fallback |

Typical artifacts

| Artifact | Filename |
|--------------------|---|
| Kindle EPUB | <code><slug>-kdp.epub</code> |
| Print PDF | <code><slug>-kdp-print.pdf</code> |
| Ebook cover copy | <code><slug>-kdp-ebook-cover.<ext></code> |
| Print cover copies | <code><slug>-kdp-print-cover-<theme>.<ext></code> |
| Metadata | <code><slug>-kdp-metadata.json</code> |

Confirm trim with **papyrus sizes** and Amazon's current KDP print specs before uploading. The next chapters document each **kdp:*** command.

kdp:ebook

Build a Kindle-oriented EPUB for Amazon KDP upload.

```
papyrus kdp:ebook  
papyrus kdp:ebook -d /path/to/book -e /path/to/out
```

Requires `kdp.ebook.enabled => true`.

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|----------------------|------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |

Output

```
export/<slug>-kdp.epub
```

Papyrus runs an internal KDP-oriented check and, when available on **PATH**, **epubcheck** (install with **brew install epubcheck**, or from [w3c/epubcheck](https://w3c.github.io/epubcheck/)).

Cover

```
'kdp' => [  
  'ebook' => [  
    'enabled' => true,  
    'cover' => 'cover-ebook.jpg', // under assets/  
  ],  
],
```

If `kdp.ebook.cover` is unset, Papyrus falls back to the light theme cover (`cover.light` or `cover.image`).

Metadata

Description falls back to "{title} - {author}" when `kdp.metadata.description` is empty. Language uses `kdp.metadata.language` when set, otherwise project `language` (default `en`).

Versus build:epub

| | <code>build:epub</code> | <code>kdp:ebook</code> |
|---------------------|--------------------------------|------------------------------------|
| Filename | <code><slug>.epub</code> | <code><slug>-kdp.epub</code> |
| Enable flag | always | <code>kdp.ebook.enabled</code> |
| Validation | packaging only | KDP checks + optional
epubcheck |
| Cover
preference | theme cover | <code>kdp.ebook.cover</code> first |

kdp:print

Build a **print interior** PDF sized for Amazon KDP: no cover page, KDP margin presets, and bleed applied around the project trim.

```
papyrus kdp:print
papyrus kdp:print --theme light
papyrus kdp:print -t dark -e /path/to/out
```

Requires `kdp.print.enabled => true`.

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|------------------------|-------------------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |
| <code>--theme</code> | <code>-t</code> | first configured theme | PDF theme for interior layout |

Output

```
export/<slug>-kdp-print.pdf
```


Print config

```
'kdp' => [
  'print' => [
    'enabled' => true,
    'bleed_mm' => 3,
    'margin_preset' => 'recommended', // or 'minimum'
    'paper' => 'white',               // cream, etc. -
  ],
  metadata only
],
```

Bleed

Bleed expands the page and margins:

- Page width / height each increase by $2 \times \text{bleed_mm}$
- Every margin increases by bleed_mm

Default `bleed_mm` is 3.

Margin presets (mm, before bleed)

| Preset | Left | Right | Top | Bottom |
|-------------|------|-------|------|--------|
| recommended | 12.7 | 9.525 | 12.7 | 12.7 |

| Preset | Left | Right | Top | Bottom |
|----------------------|-------|-------|-------|--------|
| <code>minimum</code> | 9.525 | 6.35 | 9.525 | 9.525 |

Unknown preset names fall back to `recommended`.

Paper

`print.paper` is written into the KDP metadata sidecar. It does not change PDF colour or stock — choose white vs cream in the KDP dashboard to match.

Trim size

Uses the same `document.size` / `document.format` as your regular PDFs. `papyrus doctor` warns when trim falls outside KDP bounds (width 101.6–215.9 mm, height 152.4–279.4 mm). List presets with `papyrus sizes`.

Cover

The print PDF **omits** the cover page. Export wraparound / ebook cover files with **kdp:cover**.

kdp:cover

Copy configured cover images from **assets/** into **export/** with KDP-oriented filenames. No enable flag — safe to run anytime.

```
papyrus kdp:cover  
papyrus kdp:cover -d /path/to/book -e /path/to/out
```

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|----------------------|------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |

Sources and outputs

| Source | Export name |
|---|--|
| <code>kdp.ebook.cover</code>
→
<code>assets/<file></code> | <code><slug>-kdp-ebook-cover.<ext></code> |
| <code>cover[<theme>]</code>
or <code>cover.image</code> | <code><slug>-kdp-print-cover-<theme>.<ext></code>
(one per theme) |

Missing source files are skipped (no error). If nothing is copied, the command still succeeds with a comment.

Example

```
'cover' => [
  'image' => 'cover.png',
  'light' => 'cover-light.png',
],
'kdp' => [
  'ebook' => [
    'cover' => 'cover-ebook.jpg',
  ],
],
],
```

With themes **light** and **dark**, a typical export folder might contain:

```
export/my-book-kdp-ebook-cover.jpg
export/my-book-kdp-print-cover-light.png
export/my-book-kdp-print-cover-dark.png    # if dark cover
resolves
```

Papyrus copies files as-is — it does not generate a full wraparound print cover. Use Amazon's cover calculator for spine width and bleed on the print jacket.

kdp:metadata

Emit a JSON sidecar you can keep next to KDP uploads or feed into your own publishing scripts. Always writes — no enable flag.

```
papyrus kdp:metadata  
papyrus kdp:metadata -d /path/to/book -e /path/to/out
```

Options

| Option | Short | Default | Meaning |
|-----------------------|-----------------|----------------------|------------------|
| <code>--dir</code> | <code>-d</code> | current directory | Book root |
| <code>--export</code> | <code>-e</code> | <code>export/</code> | Output directory |

Output

```
export/<slug>-kdp-metadata.json
```

Config

```
'language' => 'en',

'kdp' => [
  'print' => [
    'paper' => 'white',
    'bleed_mm' => 3,
    'margin_preset' => 'recommended',
  ],
  'metadata' => [
    'description' => 'Bookstore blurb...',
    'keywords' => ['papyrus', 'markdown', 'ebook'],
    'language' => 'en', // optional; else project language
  ],
],
```

JSON shape

```
{
  "title": "...",
  "subtitle": "...",
  "author": "...",
  "language": "en",
  "description": "...",
  "keywords": [...],
  "print": {
    "paper": "white",
    "bleed_mm": 3,
    "margin_preset": "recommended"
  }
}
```

| Field | Source |
|--|--|
| <code>title</code> / <code>subtitle</code> / <code>author</code> | Project identity |
| <code>language</code> | <code>kdp.metadata.language</code> → <code>language</code> → <code>en</code> |
| <code>description</code> | <code>kdp.metadata.description</code> (may be empty) |
| <code>keywords</code> | <code>kdp.metadata.keywords</code> (string list) |

Field**Source**

`print.*`Current `kdp.print` settings

This file is a helper for your workflow — it is not uploaded automatically to Amazon.

Migration and CI

Migrating from ibis-next

```
papyrus migrate-ibis
papyrus migrate-ibis -d /path/to/book
papyrus migrate-ibis --force
```

| Option | Short | Meaning |
|----------------------|-----------------|--|
| <code>--dir</code> | <code>-d</code> | Book root |
| <code>--force</code> | <code>-f</code> | Overwrite an existing <code>papyrus.php</code> |

This writes `papyrus.php` from `ibis.php`. If you still have custom `assets/theme*.html` files from ibis, it rewrites `<!-- IBIS:TOC -->` to `<!-- PAPYRUS:TOC -->` in those **project** files only. Bundled Papyrus themes (used when `assets/` has no theme override) already use the Papyrus marker — `migrate-ibis` does not copy or rewrite vendor assets.

After migration:

1. Remove `hi-folks/ibis-next` and any Composer patches for `ibis/mPDF`.
2. Point scripts at `papyrus` (`build`, `build:pdf`, `build:site`, ...).
3. Decide on themes: keep your ibis HTML under `assets/` as overrides, or delete those copies and use Papyrus defaults (optional `papyrus asset:publish` later if you want local copies to edit).
4. Run `papyrus doctor` and a full `papyrus build`.

Continuous integration

Copy the stub workflow from the Papyrus package:

```
stubs/github/workflows/book-build.yml
```

A typical book job installs PHP with `dom`, `gd`, `mbstring`, `zip`, `zlib`, runs `composer install`, `papyrus doctor`, optional `papyrus lint`, then `composer build`.

To publish a `build:site` output on GitHub Pages, add a job that runs `papyrus build:site` and deploys the site directory to a `gh-pages` branch (see [pages.yml](#) in this repository for a working example, including Chromium for Mermaid).

Programmatic use

```
use Milon\Papyrus\Config\Project;

$project = Project::load($bookDir);
$book =
$project->bookConverter()->convertDirectory($project->contentDir)
;
```

Checklist before release

1. `papyrus doctor` is clean
2. `papyrus build` succeeds for every theme
3. `papyrus build:site` looks right on phone and desktop (try light and dark)
4. Sample ranges still make sense after reflows (`build:sample`)
5. KDP EPUB opens in Kindle Previewer; print PDF matches trim/bleed
6. Pin a Papyrus version in the book's `composer.json`

Report problems at github.com/milon/papyrus/issues.

Command reference

Build

| Command | Purpose | Notable options |
|---------------------------|--|---|
| <code>build</code> | PDF themes, EPUB, HTML, enabled KDP | <code>-d</code> , <code>-e</code> , <code>--with-site</code> , <code>--with-sample</code> |
| <code>build:pdf</code> | PDF themes | <code>--theme</code> , <code>--parallel</code> |
| <code>build:epub</code> | EPUB3 | <code>-d</code> , <code>-e</code> |
| <code>build:html</code> | Single-file HTML | <code>-d</code> , <code>-e</code> |
| <code>build:site</code> | Multi-page HTML site | <code>-d</code> , <code>-e</code> |
| <code>build:sample</code> | Sample PDF from <code>sample.ranges</code> and/or <code>sample.chapters</code> | <code>--theme</code> |

KDP

| Command | Purpose | Notable options |
|---------------------------|---|--|
| <code>kdp</code> | All enabled KDP outputs | <code>-d</code> , <code>-e</code> |
| <code>kdp:ebook</code> | Kindle EPUB | requires <code>kdp.ebook.enabled</code> |
| <code>kdp:print</code> | Print interior PDF | <code>--theme</code> ; requires <code>kdp.print.enabled</code> |
| <code>kdp:cover</code> | Copy cover assets to <code>export/</code> | no enable flag |
| <code>kdp:metadata</code> | Metadata JSON sidecar | no enable flag |

Project tooling

| Command | Purpose | Notable options |
|----------------------------|--|---|
| <code>init</code> | Scaffold <code>papyrus.php</code> , <code>content/</code> , and an empty <code>assets/</code> | <code>--force</code> |
| <code>asset:publish</code> | Publish bundled themes, CSS, and fonts into <code>assets/</code> | <code>--force</code> , <code>--only=themes,css,fonts</code> |
| <code>doctor</code> | Validate config, paths, Mermaid, KDP trim | <code>-d</code> |
| <code>sizes</code> | List page-size presets (+ KDP in-bounds) | (no <code>-d</code> / <code>-e</code>) |
| <code>lint</code> | Lint PHP fences in <code>content/</code> | <code>--fix</code> , <code>--max-width=66</code> |
| <code>watch</code> | Rebuild via <code>build</code> on file changes | <code>--interval</code> , <code>--with-site</code> , <code>--with-sample</code> , <code>--include-drafts</code> |
| <code>serve</code> | Preview the generated site in a browser | <code>--host</code> , <code>-p/--port</code> , <code>--build</code> , <code>-s/--site</code> |
| <code>migrate-ibis</code> | <code>ibis.php</code> → <code>papyrus.php</code> ; TOC markers in local <code>assets/theme*.html</code> only | <code>--force</code> |

Shared book options

Most book commands accept:

| Option | Short | Default | Meaning |
|-------------------------------|-----------------|----------------------------------|--|
| <code>--dir</code> | <code>-d</code> | current directory | Book root
(contains <code>papyrus.php</code>) |
| <code>--export</code> | <code>-e</code> | <code><book>/export</code> | Override
export
directory |
| <code>--include-drafts</code> | | off | Include
chapters with
<code>draft: true</code> |

Also useful globally: `-v` / `--verbose` (include third-party vendor notices), `-q` / `--quiet`, `--no-ansi`.

Output filenames

| Command | Path under export |
|---------------------------|---|
| <code>build:pdf</code> | <code><slug>-<theme>.pdf</code> |
| <code>build:html</code> | <code><slug>.html</code> |
| <code>build:epub</code> | <code><slug>.epub</code> |
| <code>build:site</code> | <code><slug>-site/</code> |
| <code>build:sample</code> | <code>sample-<slug>-<theme>.pdf</code> |
| <code>kdp:ebook</code> | <code><slug>-kdp.epub</code> |
| <code>kdp:print</code> | <code><slug>-kdp-print.pdf</code> |
| <code>kdp:cover</code> | <code><slug>-kdp-ebook-cover.*, <slug>-kdp-print-cover-<theme>.*</code> |
| <code>kdp:metadata</code> | <code><slug>-kdp-metadata.json</code> |

Packagist: [milon/papyrus](https://packagist.org/packages/milon/papyrus). Source: github.com/milon/papyrus. Releases: github.com/milon/papyrus/releases.

Downloads

Preview this handbook without building locally. Files live in the Papyrus repository under [docs/](#) and update when the handbook is rebuilt.

Full PDF

| Theme | Download |
|-------|---|
| Light | <u>the-papyrus-handbook-light.pdf</u> |
| Dark | <u>the-papyrus-handbook-dark.pdf</u> |

On GitHub: [light](#) · [dark](#)

Sample PDF

A shorter preview built with `build:sample`: cover through Welcome (before the table of contents), then the `build:html` and `kdp:cover` chapters.

| Theme | Download |
|-------|--|
| Light | <u>sample-the-papyrus-handbook-light.pdf</u> |
| Dark | <u>sample-the-papyrus-handbook-dark.pdf</u> |

Configured in this book's `papyrus.php` as:

```
'sample' => [  
    'ranges' => [  
        ['from' => 1, 'to' => 5], // cover ... Welcome (before TOC)  
    ],  
    'chapters' => [  
        '09-build-html.md',  
        '15-kdp-cover.md',  
    ],  
],
```

Other previews

| Format | Link |
|------------------|---|
| This site | papyrus.milon.im |
| Single-file HTML | the-papyrus-handbook.html |

Rebuild everything from the Papyrus repo with:

```
composer build:handbook
```